Paiflow Guide

Updated Oct 30, 2026

Start with your user's problem; use problem discovery if you need help choosing one. This guide's **90/10 Split exercise is optional**. The simplest payment exercise is **Receive → Pay one recipient**. Choose a flow that fits your app.

Choose the right resource

Resource	Use it when you want to…
This guide	Build and test an XLM flow in Paiflow, then connect your own app using the SDK.
Windows Setup Guide	Install Git, Node and pnpm, run the starter, then set up Engr with Kimi on Windows.
SDK README	Build a custom Node integration with the published `@paiflow/sdk@0.2.0` beta.
SDK method and type reference	Look up SDK methods, parameters, return types and exported types.
HTTP API README	Look up endpoint schemas, errors, limits, pagination and advanced workflows.
OpenAPI JSON	Import the machine-readable API contract into development tools.
Postman collection	Try API requests with your deployment UUID and token.
AI developer context (`llms.md`)	Give your AI assistant product rules, API details and worked examples.
Starter repository	Run a minimal SDK payment example with Freighter signing, transaction status and browser recovery.

Contents

Part 1 — Build and Test Your Payment Flow

1. Prepare your testnet wallets
2. Sign in with Google and use your access code
3. Build a Receive → Split flow
4. Review and deploy
5. Make a payment through the hosted page

Part 2 — Connect Your App with the SDK

- 2.a Recommended video tutorial using the starter
- 2.b Start from a fresh repository

The written steps below belong to **2.b**.

6. Create your developer token
7. Install and configure the SDK
8. Connect to your deployment
9. Prepare and sign a payment
10. Submit and verify the payment
11. Recover an interrupted payment
12. Customise your integration
13. Troubleshoot your integration
14. Check your result

Part 1 — Build and Test Your Payment Flow

Complete the wallet, account and flow steps in Paiflow, then make a hosted-page test payment. You will use the same confirmed deployment in Part 2. The detailed amounts, wallets and receipt checks below belong to the optional 90/10 exercise. If you choose Receive → Pay instead, configure Pay to send the full amount to one recipient and verify its 1 XLM payout, with the customer paying transaction fees separately. Adapt the exercise's two-recipient checks to that single recipient.

01 Prepare your testnet wallets

Video walkthrough: Wallet setup.

1. Install the Freighter browser extension in the browser you will use for Paiflow and your local app. Complete setup and select **Stellar Testnet**.
2. Create dedicated testnet accounts for the roles below. For an easy-to-read first test, use three accounts: Customer, Shop and Community. The Shop account can also deploy the flow.
3. Fund each account with testnet XLM using Freighter's Friendbot option. Keep XLM for fees and reserves as well as the two 1 XLM test payments in this guide. Copy each account's public `G…` address and label its role.

Wallet role	What it does	Fees and signing
Deployer	Signs the deployment; may also be the Shop account	Pays deployment fees
Customer	Signs each 1 XLM test payment	Pays the payment and its transaction fee
Shop	Receives 90%: 0.9 XLM per test payment	No separate payout signature is needed
Community	Receives 10%: 0.1 XLM per test payment	No separate payout signature is needed

The two Split recipients must have distinct addresses. The customer may share a recipient account, but its net balance then includes the outgoing payment and fees; transaction evidence is clearer than a balance comparison. Use separate accounts for your first test if possible.

> Keep wallet secrets in Freighter. Never paste a secret key or recovery phrase into Paiflow, your app, an AI prompt or a repository. Testnet funds have no real-money value.

02 Sign in with Google and use your access code

Video walkthrough: Sign in with Google and use your access code.

1. Open Paiflow and choose **Continue with Google**. Use the Google account you want to use for Paiflow.
2. If this is your first signup, follow **Complete your signup** and enter the **Access code** supplied by your organiser. You can also start from **Create account** → **Continue with Google**, which asks for the code before Google sign-in.
3. Review the **Username** field. Choose a name or leave it blank to use your verified email prefix; a short suffix is added if the name is taken.
4. Submit the form to continue with Google and complete signup. If Google asks you to choose an account again, use the intended account and finish the flow.
5. When you return to Paiflow, confirm you are signed in before creating your flow. On later visits, an already linked Google account signs in without a new access code.

Access code: `bab87b6b0c4176e2963824a5751cd312`

The signup access code is different from the developer API token created in Part 2. Do not put the access code into your app configuration. Google sign-in opens your Paiflow account; Freighter separately authorises your blockchain transactions.

Checkpoint: you are signed in to your Paiflow account and can create a flow.

03 Build a Receive → Split flow

> **Optional Split exercise.** Use it to learn the mechanics, then adapt the flow to your chosen problem. The video demonstrates this specific example.

Video walkthrough: Build a Receive → Split flow.

1. Open Paiflow and sign in with your full/team account for this walkthrough.
2. Create a flow, using **On Receive** as its trigger and **Split** as its action. Connect On Receive to Split.
3. Select native **XLM** as the flow's asset. Remove unrelated blocks or default recipients from the example graph.
4. Configure two percentage recipients using your actual funded public addresses:

Recipient	Percentage	Expected share of 1 XLM
Shop	90%	0.9 XLM
Community fund	10%	0.1 XLM

The shares must total 100%. Replace every placeholder address before deploying, and check both recipient addresses against your labelled wallets.

> **Checkpoint:** your graph has one trigger, a connected Split action, XLM throughout and two real recipients. You are modelling one shop's payment rules, not a separate contract for each customer.

04 Review and deploy

Video walkthrough: Review and deploy.

1. Open the deployment review and read the **English preview**. Confirm that it describes receiving XLM and paying your two recipients at the intended percentages.
2. Connect the funded deployer wallet. Confirm Freighter is on Testnet.
3. Start deployment. In Freighter, review the network, source account, requested contract operations and displayed fee before confirming.
4. Wait for the deployment to report **CONFIRMED**, then open its deployment page.

Copy the deployment's UUID. It is different from the flow ID and the Stellar contract address beginning with `C`. API calls use the deployment UUID.

> **Checkpoint:** you have a confirmed deployment with the correct asset and recipients.

05 Make a payment through the hosted page

Video walkthrough: Make a payment through the hosted page.

Before you pay: confirm **Testnet**, the Customer wallet address, **XLM**, **1 XLM**, the configured Shop/Community recipients and the displayed transaction fee. Apply the same checks when paying from your app later. If the fee is unexpected, stop and ask support rather than accepting it to finish the exercise.

1. Open the deployment's hosted payment page or QR link. For this Receive flow, its path is `/trigger/YOUR_DEPLOYMENT_UUID` on the Paiflow site.
2. Connect your funded customer wallet. If you changed accounts in Freighter, check that the app shows the intended customer's public address.
3. Enter **1 XLM**, review the payment and approve the Testnet transaction in Freighter.
4. Wait for confirmation. Inspect the deployment's live activity and transaction evidence for the split payout.

For this example, the payment should route **0.9 XLM to the shop and 0.1 XLM to the community fund**. Fees are additional to the customer's 1 XLM payment. If a recipient is also the paying wallet, its net balance change will include both the payment and fees; use the transaction's payout evidence to check the split.

Checkpoint: save the payment transaction hash and verify its expected payouts. A slow live feed is not proof of failure; check the existing transaction before paying again. You can look up the hash on Stellar Expert Testnet.

Part 2 — Connect Your App with the SDK

2.a Recommended video tutorial using the starter

> **We recommend this path.**
>
> The tutorial is in the video below and uses the Paiflow starter repository, which already includes the payment integration. Follow the video to set up the working example, then adapt it to your app.

Video walkthrough: Connect your app using the starter repository.

2.b Start from a fresh repository

> **Follow the written steps below if you are starting from a fresh repository.**
>
> This path shows you how to add the Paiflow SDK and build the payment integration yourself. It does not use the starter repository.

Connect the deployment from Part 1 to your own app. The Node SDK runs on your server; the customer's wallet signs in the browser. The browser talks to your app's server, which keeps the developer token private.

06 Create your developer token

1. Sign in as the owner of your confirmed deployment and open **Developer API**.
2. Under **Create developer token**, enter an optional label and choose an expiry that covers your integration work.
3. Choose **Read and operate**. Read only allows discovery and reads, but cannot prepare or submit this payment.
4. Select the deployment from Part 1. It must be confirmed, active and owned by your account. Create the token and copy its one-time reveal into your server configuration before leaving or hiding the page.
5. Keep the deployment UUID with your token. You can edit the selected deployments later; a newly created deployment does not receive access automatically.

Value	How to recognise it
Deployment UUID	UUID for the confirmed deployment; not the flow ID or contract address
Developer token	Begins with `pfd_`; covers explicitly selected deployments
Deployment token	Begins with `pfk_`; the supported single-deployment alternative from **API access**
Customer address	Public `G…` address; comes from the customer's connected wallet

> **Keep the API token server-side.** Do not put it in browser code, `NEXT_PUBLIC_` variables, screenshots or public repositories. If exposed, revoke it and issue a replacement. Token permission does not replace wallet consent; your server never needs customer secret keys.

See the developer-token guide for permissions and management. Changing a token's label, permission or expiry requires a replacement token; deployment selections can be edited.

07 Install and configure the SDK

> Use an existing Node/TypeScript app with **Node** `⩾22.22.2 <23`, the runtime supported by `@paiflow/sdk@0.2.0`. The SDK uses ESM; a plain Node app needs `"type": "module"`, and TypeScript consumers use `NodeNext` module resolution. Your framework can provide its own compatible server setup.

Install the published SDK beta in your app:

```
pnpm add @paiflow/sdk@0.2.0
```

Pin `0.2.0` for this walkthrough. The `beta` tag points to 0.2.0; unversioned installation uses `latest`, which still points to 0.1.0 and lacks developer-token discovery.

Use the package manager already chosen by your app. For the browser wallet integration below, install `@stellar/freighter-api` through your app's bundler. The complete server example separately uses `@stellar/stellar-sdk` to compare the signed and prepared transaction bodies; this is an application dependency, not a core SDK dependency.

Load these values through your app's server configuration. If your framework uses `.env.local`, put them there; a plain Node process must explicitly load its environment file.

```
PAIFLOW_BASE_URL=https://beta.app.paiflow.xyz
PAIFLOW_DEPLOYMENT_ID=YOUR_CONFIRMED_DEPLOYMENT_UUID
PAIFLOW_API_TOKEN=YOUR_DEVELOPER_API_TOKEN
```

Use the origin supplied for your environment, your deployment UUID and a token explicitly authorised for it. Keep the file ignored by git. Do not use `NEXT_PUBLIC_` or send the token to the browser. Restart your server after changing configuration. The SDK payment example uses these same server settings; complete configuration enables payments.

08 Connect to your deployment

Keep SDK imports in a server-only module. Read `paiflowOrigin`, `developerToken` and `deploymentId` from the trusted configuration above, not from browser input:

```
import { Paiflow } from "@paiflow/sdk";

const client = new Paiflow({
  baseUrl: paiflowOrigin,
  token: developerToken,
});
let selected = false;
for await (const page of client.deployments.pages({ limit: 20 }, { maxPages: 100 })) {
  if (page.data.items.some((item) => item.id === deploymentId)) {
    selected = true;
    break;
  }
}
if (!selected) throw new Error("Check the configured deployment and token grants");
const deployment = client.deployment(deploymentId);
const { data: capabilities } = await deployment.capabilities.get();
if (capabilities.network !== "testnet") {
  throw new Error("This walkthrough requires testnet");
}
```

Keep `deploymentId` fixed in trusted server configuration; discovery does not choose or grant deployments for your app. Selection changes can remove access after discovery. The two-deployment example shows optional multi-deployment use.

Check `capabilities.operations.execute` for your Receive flow, then use preparation simulation to check current eligibility. Capability metadata alone does not establish available funds or successful execution. SDK methods return `{ data, metadata }`; keep request/history IDs for recovery without exposing credentials.

> **Checkpoint:** your backend can read the intended testnet deployment without sending its token to the browser.

09 Prepare and sign a payment

Use the server example and browser example as adapters for your own app. They are framework-neutral building blocks, not a ready-to-run checkout. Bind the server handlers to your own `/checkout/prepare` and `/checkout/submit` routes, and mount the browser adapter on your payment UI.

Supply the example's application dependencies: authenticated users and verified wallet ownership, fixed app origin, a durable order store/lock, configured payment description, an explicit payment-review UI and a receipt verifier. Apply rate limits, validate requests and return safe errors in your route adapter. Do not use the examples' local test fixtures as production storage.

> Your authenticated prepare handler derives the customer wallet and price from its trusted order record. Bind that order to its intended deployment UUID and server network; reject attempts to use it with another deployment or network. For the walkthrough, the order price is **1 XLM**, represented as `"10000000"` stroops. Its SDK call is:

```
const prepared = await deployment.execute.prepare({
  from: order.walletAddress,
  amount: order.amountStroops,
});
```

1. Save the original preparation to that order **before** asking the wallet to sign. A repeated request for the same order returns that preparation, not a new payment.
2. Send the preparation's unsigned XDR, server network/passphrase and trusted order-review details to your browser UI. Never send the API token.
3. Connect Freighter to your app's origin. Confirm the connected customer matches the order wallet. Show **TESTNET**, the 1 XLM price, the configured recipients and displayed fee; ask for explicit consent.
4. The browser adapter calls Freighter's `signTransaction` with the original XDR and **the preparation's `networkPassphrase`**. The SDK stays on the server; the browser can import SDK types only.
5. Send the signed envelope to your own submit route. Before SDK submission, compare its transaction-body hash to the saved original under that server passphrase, as `assertOriginalEnvelope` does in the server example. Reject a changed body or fee-bump wrapper; added signatures are allowed.

> An ordinary preparation lasts approximately **180 seconds**. Unlock Freighter before paying. Expiry, cancellation or an interrupted signing response must retain the original record for recovery; do not silently prepare a replacement. The SDK's optional signing helper does not itself parse or bind the signed transaction body.

10 Submit and verify the payment

> Your submit handler rechecks the authenticated order owner, deployment, network and original preparation. Persist the original signed envelope and its transaction hash in access-restricted storage **before** sending it. For the same order, never replace an already saved submission with a different envelope.

The server example submits through the SDK with the original preparation metadata:

```
const submitted = await deployment.execute.submit(
  { signedXdr: order.signedXdr },
  {
    wait: false,
    preparationRequestId: order.prepared.metadata.historyRequestId,
  },
);
```

`wait: false` can return `PENDING`; an HTTP 200 can also carry a failure. Preserve the original hash and query it explicitly:

```
const receipt = await deployment.transactions.get(order.txHash);
const events = await deployment.events.list({
  txHash: order.txHash,
  limit: 100,
});
```

> Use your receipt verifier to establish final **SUCCESS**, the intended deployment/trigger, original hash, payer, asset, exact order amount and configured payouts. For this flow, verify **0.9 XLM to the shop and 0.1 XLM to the community fund**. Events can lag; follow `nextCursor`/`hasMore` when paginating and check chain transfer evidence if needed. Missing events do not authorise another payment.

Mark the order paid only after that verification. Show success, pending or a safe error in your UI. Do not fulfil an order just because submission returned HTTP 200. Keep the payment record and hash so a recovery worker can run the same verifier later.

> **Checkpoint:** your app has made a 1 XLM payment through the SDK and verified both expected payouts.

11 Recover an interrupted payment

> The SDK does not automatically retry writes, poll finality or reconnect your payment UI. Implement a **Check payment** action backed by your server's saved order, preparation, envelope and hash. Protect recovery endpoints with the same user/ownership checks as checkout.

Observed result	What your app should do
SUCCESS with complete matching payout evidence	Mark the existing order paid; do not send it again.
PENDING, UNKNOWN, REJECTED, timeout or submission error	Keep the original record and check the original hash. An error does not prove nothing was broadcast.
Confirmed FAILED	Retain the failed receipt and resolve its cause before a deliberate new payment.
Expired preparation, missing signature or NOT_FOUND	Retain the record and ask support if finality cannot be established. Clock expiry and missing history do not prove safe replacement.
Missing or delayed events	Keep the order pending verification and query existing evidence; do not create another payment.

Revoking a token, expiry or removing a deployment grant blocks later requests; it does not cancel an already submitted transaction. Keep the original order and recover its hash using a replacement credential explicitly authorised for the same deployment, or the owner's supported dashboard recovery path. Never move the order to another deployment to clear an access error.

The published execute API cannot prove ledger-covered noninclusion from expiry or NOT_FOUND alone. Do not reset the order or create a replacement merely to clear an error. An explicit recovery path may reuse the original signed envelope; it must never invent a new one or change the original order.

Keep recovery state on the server so reloading the browser does not lose the payment. A signing response can be lost even when a wallet has acted, so absence of a saved signature is not proof that nothing happened. Never log tokens, wallet secrets or signed envelopes. Ask support using the deployment UUID, hash and request ID, not the signed transaction.

12 Customise your integration

Keep preparation, wallet consent, submission and receipt verification intact while replacing the customer interface with your checkout, dues, donation or other app. Associate events by transaction hash and deduplicate by `eventId`. Authorise business actions in your app; a API token alone is not permission for any visitor to release funds.

The SDK and API use integer stroop strings. Convert decimal UI amounts using bigint, not floating-point arithmetic: 1 XLM is `"10000000"`. The builder uses percentages; graph/API shares use basis points: 90% is `9000`, 10% is `1000` and the total is `10000`.

For detailed method shapes, SDK errors and advanced workflows, use the SDK README. Direct HTTP integrations can use the API README. Schedules, subscriptions, escrow and callbacks have different funding, consent and execution requirements; confirm the actual deployment's capabilities and live state before choosing one.

Optional: start from a ready-made app

The SDK payment example provides a runnable 1-XLM native-testnet payment with Freighter signing, original-transaction status and browser recovery. It accepts `pfd_` or `pfk_` credentials for one explicitly configured deployment. Follow its README; funding, Schedule, Subscription and activity panels are outside its minimal scope.

Other optional app ideas include workshop ticketing or service booking with Receive → Pay, donation routing with an amount condition, or a shared savings lock with a time condition and Hold. Confirm support for conditions and release operations with organisers before choosing them. The team builds the app's ticketing, booking and other features; a deposit held for later release is not a verified payout. These examples are inspiration, not assigned challenges. See previous hackathon winners for more ideas.

Campus Snacks is the sample for a server-priced USDC shop and 90/10 split. Follow its own asset and configuration instructions. Preserve the original-envelope recovery when adapting either example; neither supplies production authentication or order fulfilment.

Optional: use USDC

Finish the XLM exercise first. A USDC flow needs a new deployment configured for that asset; the XLM deployment above does not switch assets when you change an app amount field.

1. In every customer and recipient wallet, select Testnet and add a trustline for asset code `USDC`, issuer `GBBD47IF6LWK7P7MDEVSCWR7DPUWV3NY3DTQEVFL4NAT4AQH3ZLLFLA5`. A trustline enables receipt; it does not provide a balance.
2. Obtain testnet USDC for the customer, for example from the Circle testnet faucet where Stellar Testnet is available. Confirm the exact asset and balance. Keep XLM for reserves and fees; Friendbot provides XLM, not USDC. A token swap is not a prerequisite for this guide.
3. Build a separate Receive USDC → Split flow with the same percentage rules and funded recipient accounts. Review the correct asset and issuer, deploy and explicitly add that deployment to your developer token's selection (or create its own `pfk_` token).
4. Resolve any pending XLM operation before changing your app configuration. Configure the SDK backend with the new deployment UUID and an authorised token, restart, and verify a small USDC payment. A 1 USDC payment should pay 0.9 USDC and 0.1 USDC, with XLM fees paid separately.

Never use mainnet funds for this exercise. If simulation reports a missing trustline, fix the recipient's trustline for the exact issuer; do not substitute an unrelated token named USDC.

13 Troubleshoot your integration

Symptom	What to check
Missing or rejected signup access code	Ask your organiser for an active code; enter it in Paiflow signup, not the SDK configuration.
SDK engine/import error	Use Node `⩾22.22.2 <23` and compatible ESM/TypeScript settings.
Freighter unavailable or wrong customer	Unlock it, approve this app's origin and check the wallet against the order's verified address.
401 or 404	Check origin, deployment UUID, active service, token expiry/revocation and explicit deployment selection.
403 while preparing or submitting	Use a developer token with **Read and operate**; Read only cannot make this payment.
`client.deployments` **is unavailable**	Upgrade to `@paiflow/sdk@0.2.0`; registry 0.1.0 lacks the new discovery methods.
Insufficient funds or missing account	Fund the customer and recipients on testnet; leave XLM for fees and reserves.
Missing USDC trustline	Check the exact issuer and trustline on every recipient, plus the customer's USDC balance.
422 validation error	Inspect the safe message; check the input asset, amount, recipients and executable trigger.
429 rate limit	Respect Retry-After; each endpoint allowance is shared across the token's selected deployments.
PENDING, expiry, timeout or submission error	Recover the saved original transaction; do not create a new preparation or discard the order.
Browser reload during signing/submission	Load the existing order from your server and check its hash; never treat reload as permission to pay again.
FAILED	Keep the confirmed failure and fix its cause before a deliberate new attempt.
Confirmed transaction, delayed feed	Verify the original hash and payouts while the indexed feed catches up.

When asking for help, include the deployment UUID, transaction hash, safe error code/message and request ID if available. Omit credentials and signed envelopes.

14 Check your result

The developer-token hosted report verifies native payments and replacement-token recovery; it does not certify this exact 90/10 Split walkthrough in your target environment. Complete the checks below in your target environment.

Your end-to-end integration is ready when you have:

- A confirmed XLM Receive → Split deployment with your two recipient addresses.
- A hosted-page payment from Part 1 with matching 0.9/0.1 XLM payouts.
- An SDK-driven app payment from Part 2 with independently verified payouts.
- The intended deployment explicitly selected on a Read and operate developer token, or its own single-deployment token.
- The deployment UUID, server network, original order/payment records and hashes retained.
- Server-only API credentials, wallet-only signing and no customer secret keys in your app.
- A clear success/pending/failure display and an original-transaction recovery path.